Recitation 3: Big-O and Computational Complexity

Justin

November 2026

1 Big-O

True or False? Give a brief justification for each.

1. $n \log n \in O(n)$
2. $10^9 n \in \Theta(n)$
3. $\log(n^4) = O(\log n)$
4. $n^6 \cdot 2^{-n} \in O(1)$
5. $3^n \in O(4^n)$
6. $2^{\sqrt{n}} \in O(n^3)$
7. $2^n \in \Theta(n^{1,000,000})$
8. $n^k \in O(k^n)$ for $k \geq 2$

Solutions

1. **False.** For large n, $\log n > 1$, hence $n \log n > n$.
2. **True.** Multiplying by a constant preserves $\Theta(\cdot)$.
3. **True.** $\log(n^4) = 4 \log n = O(\log n)$.
4. **True.** $n^6/2^n \to 0$; in particular it is bounded, so $O(1)$.
5. **True.** $3^n \leq c \times 4^n \to (\frac{3}{4})^n \leq c$ which is true since $\frac{3}{4} < 1$.
6. **False.** Take logs: compare $\sqrt{n}$ vs $3 \log n$; $\sqrt{n}$ dominates, so $2^{\sqrt{n}}$ eventually outgrows n^3.
7. **False.** Exponential 2^n dominates any polynomial, regardless of the polynomial's degree Also, $\lim_{n\to\infty} \frac{n^{1,000,000}}{2^n} = 0$.
8. **True** Exponential k^n dominates any polynomial n^k. Take logs: $\log(n^k) = k \log(n) \leq n \log(k) = \log(k^n)$.

Formally prove the following statements:

1. $3n^2 + 7n \log n + 12n + 5 \in O(n^2)$

2. $n^2 + 3/n \in O(n^2)$

3. $n^2 + 3n \log n \in O(n^2)$

4. $n \log n \in O(n^{1+\epsilon})$ for every $\epsilon > 0$

Solutions.

1. **Proof.** If we assume $n_0 = 1$, then $7n^2 > 7n \log n$, $12n^2 > 12n$ and $5n^2 > 5$. Thus, $3n^2 + 7n \log n + 12n + 5 \leq 3n^2 + 7n^2 + 12n^2 + 5n^2 = 27n^2$. Take $c = 27$, $n_0 = 1$.

2. **Proof.** For $n \geq 1$, $3/n \leq 3 \leq 3n^2$. Hence $n^2 + 3/n \leq n^2 + 3 \leq 4n^2$. Take $c = 4, n_0 = 1$.

3. **Proof.** For $n \geq 2$, $\log n \leq n$, so $3n \log n \leq 3n^2$. Thus $n^2 + 3n \log n \leq 4n^2$ for $n \geq 2$. Take $c = 4, n_0 = 2$.

4. **Proof.** Fix $\epsilon > 0$. Note that $\lim_{n\to\infty} \frac{\log n}{n^\epsilon} = 0$, so $\exists n_0$ s.t. $\log n \leq n^\epsilon$ for $n \geq n_0$. Then $n \log n \leq n \cdot n^\epsilon = n^{1+\epsilon}$ for $n \geq n_0$, i.e. $n \log n \in O(n^{1+\epsilon})$.

2 Computational Complexity

1. Analyze the running time of the following program:

```
v = 0
for i = 1 to n do
    for j = 1 to n do
        v = i · j^2
    end for
end for
```

Solution. The body runs once per pair (i, j): exactly n^2 executions of an $O(1)$ statement. Total time $\Theta(n^2)$; total body executions $= n^2$.

2. Analyze the running time of the following program (you may assume that A is already sorted):

```
i, j = 0, length(A) − 1
while i < j do
    if A[i] ! = A[j] then
        return False
    end if
    i = i + 1
    j = j − 1
end while
return True
```

Solution. Although are two variables that are incrementing for every pass of the while loop, the total number of increments for both variables combined is never $> len(A)$. Hence, the runtime is $\Theta(n)$ where $n = len(A)$.

3. Suppose the function $F(i, j)$ takes $\Theta(i)$ time to execute. Analyze the runtime of the following program, which has input n:

```
for i = 1 to n do
    for j = i to n do
```

```
        Call F(i, j)
    end for
end for
```

Hint: $\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6}$

Solution. Let $T(n)$ be the running time of the algorithm. Since $F(i,j) = \Theta(i)$, summing over the two loops gives $T(n) = \sum_{i=1}^{n} \sum_{j=i}^{n} \Theta(i) = \sum_{i=1}^{n} (n-i+1)\Theta(i) = \Theta(\big(\sum_{i=1}^{n} i(n-i+1)\big)$. Then,

$$\sum_{i=1}^{n} i(n+1) - i^2 = \frac{n(n+1)}{2}(n+1) - \frac{n(n+1)(2n+1)}{6} = \frac{n(n+1)(n+2)}{6} = \Theta(n^3).$$

Hence the runtime is $\Theta(n^3)$.

4. Given an array of integers *nums* and an integer target *num*, return indices of the two numbers such that they add up to *target*. You may assume that each input would have exactly one solution, and you may not use the same element twice. What is the runtime of your solution? Can you come up with a solution with a runtime of $\Theta(n)$?

   Solution.

```
seen ← {}
n ← length(nums)
for i = 1 to n do
    num ← nums[i]
    complement ← target − num
    if complement ∈ seen then
        return [seen[complement], i]
    end if
    seen[num] ← i
end for
return []
```

   This solution has a runtime of $\Theta(n)$. Now, compare this solution to the brute-force solution:

```
n ← length(nums)
for i = 1 to n do
    for j = i + 1 to n do
        if nums[i] + nums[j] == target then
            return [i, j]
        end if
    end for
end for
return
```

   What is the runtime of this second approach? Why is the other approach better?